

AML-2404 AI and ML Lab Lambton College	Capstone Project Weekly Progress Report Week 4 Report
---	--

Semester	2026
Course Code	AML-2404
Project Title	AI-Powered Customer Service Chatbot Using Python and NLTK
Group Name	Dev Pankajkumar Bajaniya, Jatin Vaghela, Pratikkumar Bhupendrabhai Chavda, Amal Satheesan, Karop Dezosa Sebastian
Student Names / IDs	Dev Jatin Pratik Amal Dezosa
Reporting Week	Week 4
Faculty Supervisor	William Pourmajidi
Team Lead this Week	Jatin Vaghela

Grade the level of collaboration between team members (to be done by Team Lead):

(use: good, medium, below expectation)

Dev	Jatin	Pratik	Amal	Dezosa
Good	Good	Good	Good	Good

Progress Made in Reporting Week at Individual and Group Level

(Detailed progress for Week 4 — continued from Week 3 milestones; team pivoted to realistic MVP scope)

Individual Progress:

Dev (Backend Developer):

- Migrated the Flask backend to support the new React frontend, enabling full end-to-end REST API communication between the chat UI and NLP engine.
- Integrated database-backed chat logging (replacing JSON file storage) with session ID tracking for each conversation.
- Updated API endpoints (/chat, /logs, /stats, /models, /business-info) and verified CORS configuration for React integration.
- Fixed the FAQ data loading bug in chatbot.py where business_data.json structure caused initialization errors on first request.
- Implemented session-based message persistence so chat history is stored and retrievable per user session.

Jatin (AI / ML Specialist):

- Expanded intents.json with additional training patterns across underrepresented intent categories (payment disputes, account recovery, complaints).
- Significantly expanded business_data.json with more products, policies, FAQs (10 → 25+), and customer support information.
- Updated the Ollama AI prompt to include expanded business context (products, policies, company info) for more accurate AI-generated responses.
- Conducted intent matching evaluation using docs/sample_test_questions.md; documented pass/fail results for all 12 intent categories.
- Tuned and documented the pattern-matching confidence threshold for consistent intent recognition.

Pratik (Database Manager):

- Designed and implemented the database schema for conversation logs, user sessions, and message analytics.
- Integrated the database layer with the Flask backend; verified read/write operations for chat history and session tracking.
- Built analytics queries for most common intents, fallback trigger rate, and model usage statistics (FAQ vs pattern vs Ollama).
- Migrated existing chat log data from chat_logs.json to the database with data integrity verified.
- Set up indexing on session ID and timestamp fields for efficient query performance.

Amal (Frontend / UI Designer):

- Built a fully functional React chat interface (Vite + React) replacing the original HTML/vanilla JavaScript frontend.
- Implemented chat components: message bubbles, typing indicator, model selector, AI mode toggle, and clear chat functionality.
- Added message timestamps, intent badges, confidence scores, and model-used indicators to bot responses.
- Completed mobile-responsive layout with breakpoints for phone and tablet screen sizes.
- Applied a professional UI design with improved colors, spacing, typography, and error/loading states.

Dezosa (Documentation & Deployment):

- Updated all project documentation to reflect the Week 4 pivot: React frontend, database logging, expanded business data.
- Documented the revised tech stack honestly (Python, Flask, NLTK, Ollama, React, Database) and removed overstated features from prior reports.
- Created setup guide covering React frontend, Flask backend, database, and Ollama installation steps.
- Prepared test results documentation and a 5-minute live demo script for the final presentation.
- Coordinated team pull requests and resolved merge conflicts between frontend, backend, and database branches.

Team Progress:

- Successfully completed the React UI Integration milestone — chatbot runs end-to-end via React frontend, Flask backend, and database logging.
- Database layer is live — all chat interactions are persisted with session tracking for analytics and history.
- Business knowledge base expanded — business_data.json now contains 25+ FAQs, additional products, and complete policy information.

- Project scope realigned — team removed over-engineered features (WebSockets, ML classifier training, cloud deployment) and focused on a realistic, demo-ready MVP.
- All 12 intent categories tested with documented results; hybrid system (FAQ → pattern matching → Ollama AI) verified working.
- Project is presentation-ready with live demo environment configured on local machine.

Areas / Tasks Where Progress Could Not Be Made or Difficulties Encountered

Individual Blockers:

Dev:

- Initial CORS issues between React dev server (port 5173) and Flask (port 5000) required additional header configuration; resolved.
- Database integration required refactoring existing log save/load functions in app.py; completed without breaking existing API contracts.

Jatin:

- Expanding business data while keeping JSON structure consistent with the FAQ cache loader required careful formatting; one structural bug was found and fixed during testing.
- Ollama response times (1–3 seconds) for complex queries remain slower than FAQ/pattern responses; acceptable for demo purposes.

Pratik:

- Database choice (SQLite vs MongoDB) required team discussion; final implementation completed after evaluating setup complexity for the capstone timeline.
- Ensuring backward compatibility with existing /logs and /stats API responses during migration took additional testing time.

Amal:

- Migrating all features from the vanilla JS frontend (model selector, AI toggle, typing indicator, badges) to React components required more time than estimated.
- Minor layout adjustments still needed on very small screens (< 360px width); functional but not pixel-perfect.

Dezosa:

- Correcting prior report inaccuracies (features listed but not implemented) required careful rewriting of documentation to match actual project state.
- No major technical blockers this week.

Team Blockers:

- Team formally decided to drop WebSocket, ML classifier training, sentiment analysis, Docker deployment, and cloud hosting from scope — these were listed in earlier reports but were not feasible within the capstone timeline with local Ollama dependency.
- Multi-turn conversation context (follow-up questions referencing earlier messages) is partially implemented via session history storage; full context injection into Ollama prompts is planned as a final enhancement before presentation.

Tasks to be Completed Next Week (Week 5 / Final Week) at Individual and Group Level

Individual Tasks:

Dev:

- Implement basic multi-turn context by passing last 2–3 session messages into the Ollama prompt.
- Final API stability testing and error handling improvements before presentation.

Jatin:

- Run final round of intent testing on all 12 categories; fix any remaining pattern gaps.
- Prepare NLP/AI section of the final project report with accurate system architecture description.

Pratik:

- Finalize analytics dashboard or summary view showing intent distribution and chat statistics.
- Verify database integrity with full conversation test runs.

Amal:

- Final UI polish pass based on team review feedback.
- Ensure demo UI works smoothly on professor's presentation screen resolution.

Dezosa:

- Submit final project report to Professor William Pourmajidi.
- Lead final presentation rehearsal with live chatbot demonstration.

Team Tasks:

- Run full end-to-end testing: all 12 intents, FAQ matching, Ollama AI responses, edge cases (empty input, ambiguous input, long queries).
- Complete final presentation rehearsal with live demo (React UI → Flask API → NLTK/Ollama → Database logging).
- Submit final project report and prepare Q&A responses for faculty review.

Project Completed Percentage (Estimate): 88% — On Schedule for Final Presentation